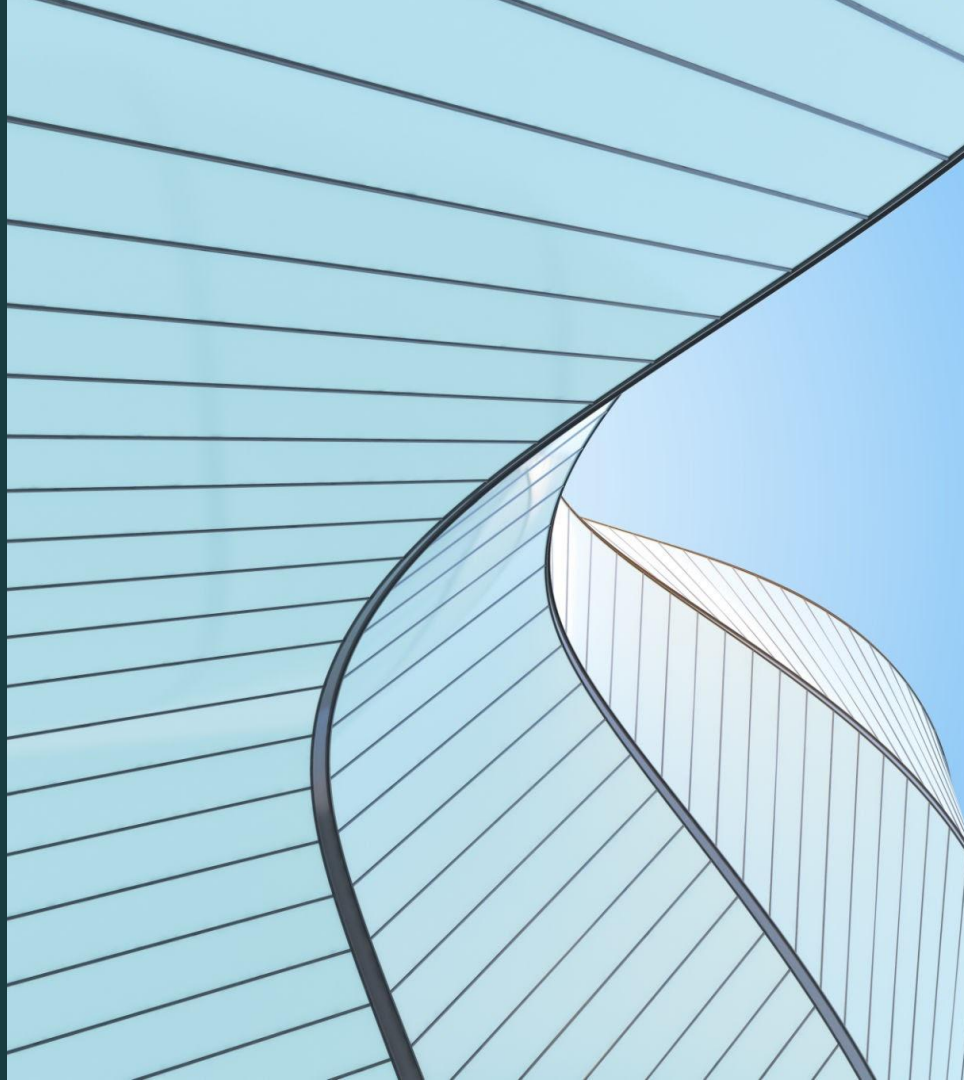
Spherical Easel Sprint 2 Presentation



Team:

Wyatt Watson, Sam Rios, Ben Turner, Jason Bos



Project Summary

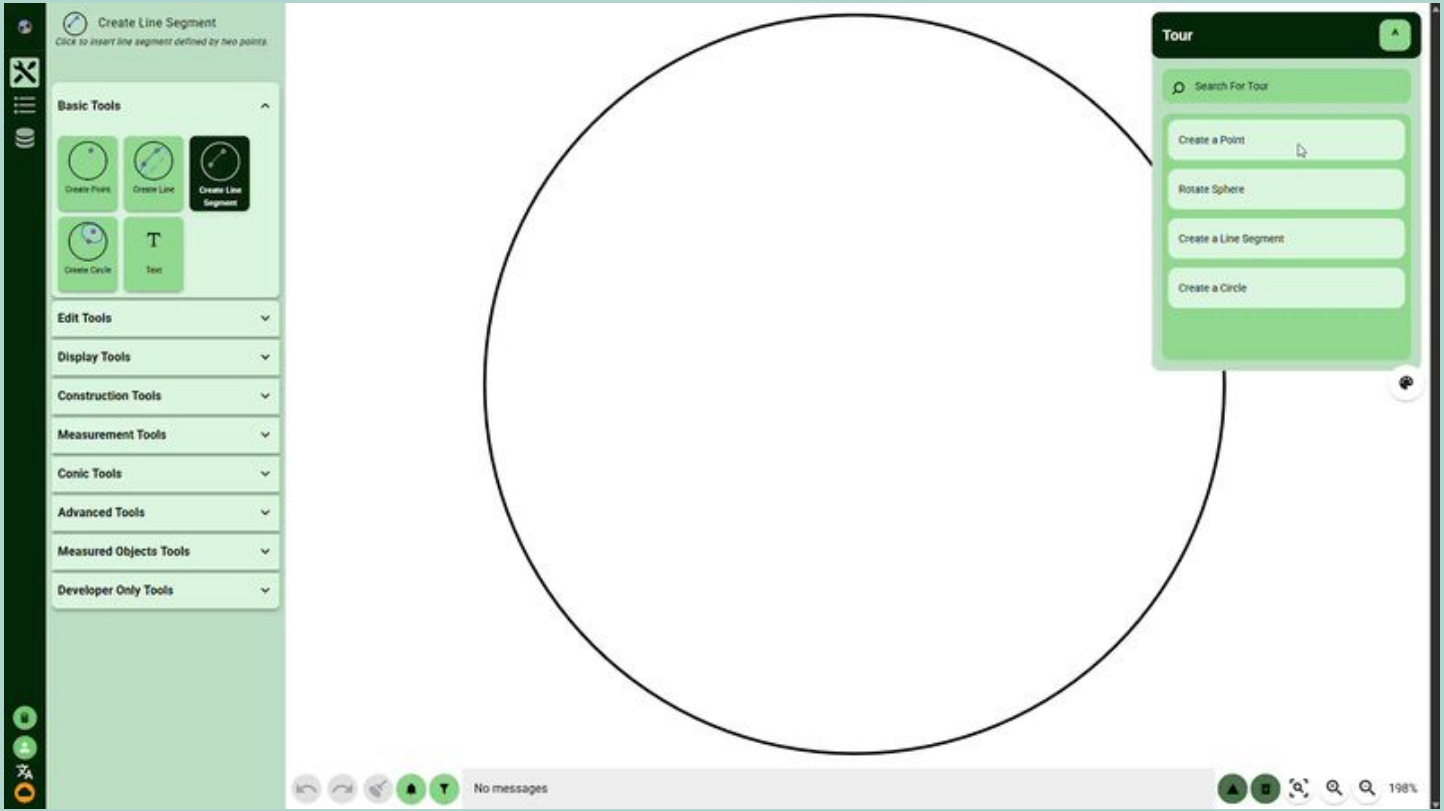
Spherical Easel is an interactive application for exploring spherical geometry. Our project aims to improve the user experience by adding guided, step-by-step tours that use Vue.js and Typescript to help new users learn the tools and navigate the application more easily.

Progress So far...

Sprint 1 : Experimenting and getting familiar with codebase
Week 1: Development environment setup, codebase exploration, GitHub/Linear workflow
Week 2: Tour prototypes and comparison of implementation approaches

Sprint 2 : Developing foundation iteration, and first roll outs.
Week 1: Tour structure, data model, and framework design
Week 2: Basic reusable tour framework and first working tour

Layout and Functionality Overview



Intended and Actual Progress - Wyatt

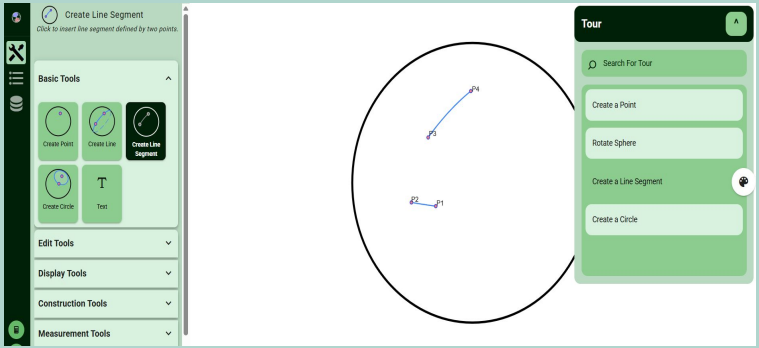
Progress:

- **Tour File System**
  - Organized File Structure
  > UITour.vue: Capture User input/ UI Display
  > Functions.vue: Store Tour Functions/ Logic

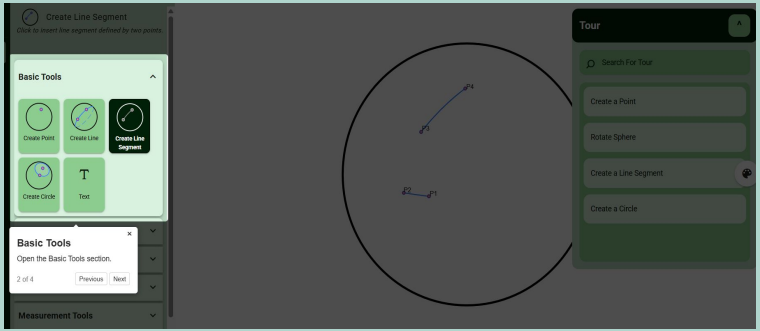
- **First Version of the Tour-Tab**
  - Completed Reusable Functions
  - Functional Basic Tours

- **User Object Detection Prototypes**
  - Observed and Captured Pinia State Changes for conditional tours.

UITour.vue: User Input / Ui



Functions.vue: Tour Functions/Logic



Intended and Actual Progress - Sam

Progress:



- **Implementation of Driver.js**
  - Found a JS package to highlight HTML elements.
  - Found UI Elements for Driver.js System in tours.
  - Integrated Driver.js into Tour Logic.

- **Built Robust ID System of UI Elements**
  - Created a naming system for getting correct element
  - Added attribute ‘data-tour’ to HTML elements for highlighting

- **Creating Tours (In-Progress)**
  - Created the first couple of tours using Driver.js
  - Continuing to create more tours using new naming system

Element Naming Properties

Tab: data-tour='ta-...' - a tab that will have different groups.
Group: data-tour='gr-...' - a “**Group**” tab that needs to be clicked/opened to access buttons
Button: data-tour='bu-...' - a **button** that we want to highlight
'[data-tour="attribute"]' - This is how you highlight the element in Driver.js
The **groups** are all assigned a name based on their **group name** in toolgroups.ts
The **buttons** are all assigned a name based on their **action** in tooldictionary.ts

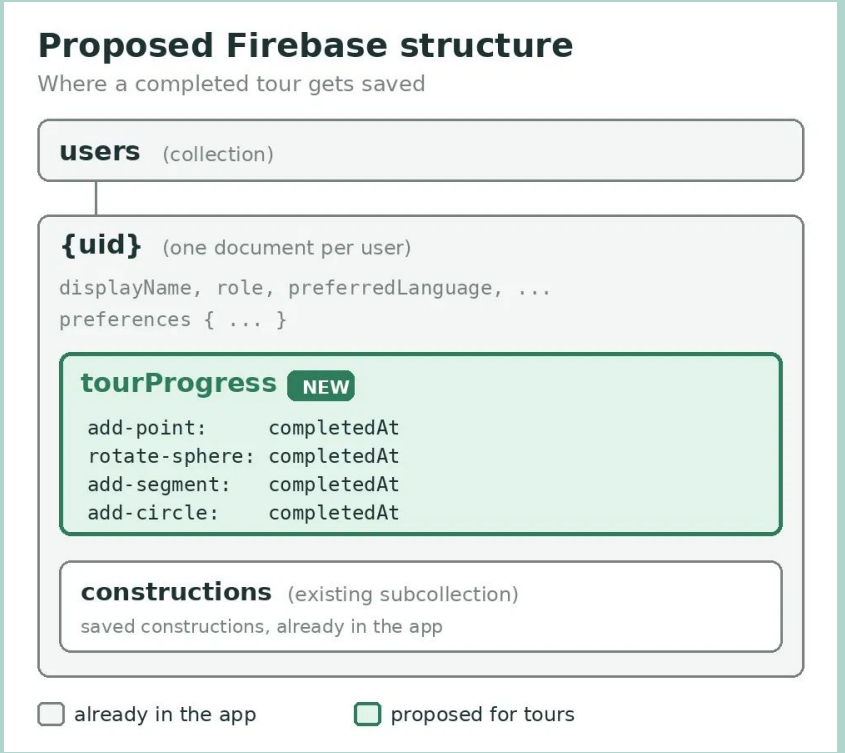
Element Name	Attribute
Sphere Canvas	#sphereContainer
Tools Tab	**data-tour='ta-tools'**
Basic Tools Group	data-tour='gr-BasicTools'
Create Point	data-tour='bu-point'
Create Line	data-tour='bu-line'
Create Line Segment Button	data-tour='bu-segment'
Create Circle Button	data-tour='bu-circle'
Text Button	data-tour='bu-text'
Edit Tools Group	data-tour='gr-EditTools'
Select Objects Button	data-tour='bu-select'
Delete Button	data-tour='bu-delete'
Display Tools Group	data-tour='gr-DisplayTools'
Hide Objects	data-tour='bu-hide'
Toggle Label Display	data-tour='bu-toggleLabelDisplay'
Move Objects	data-tour='bu-move'
Rotate Sphere	data-tour='bu-rotate'

Intended and Actual Progress - Ben

Progress:

- **Investigate how the app already uses Firebase**
  - User data lives in Firestore, one document per user (users/{uid})
  - Saved constructions already use a subcollection under each user

- **Proposed structure for tour completion**
  - New tourProgress field on the user document (add-point, rotate-sphere, ...)
  - Matches how preferences are already stored inside the user document
  - Comes back with the profile the app already loads at login



Status: design only, not connected to the tours yet

Intended and Actual Progress - Jason

Progress:

- **Investigate Logical Checks in Tour-System**
  - Driver.js Advance on Click
  - Check if certain object was created by user
  - Use onNextClick to stop users from advancing until they have completed the step

```
const driverObj = driver({
  advanceOnClick: true,
  showProgress: true,
  steps: [
    { element: '#pick-plan', popover: { title: 'Pick a Plan',
    { element: '#billing-toggle', popover: { title: 'Billing',
    { element: '#checkout-btn', popover: { title: 'Checkout',
  ],
});
// Hooks to run on button clicks. Each hook receives
// the following parameters:
//   - element: The current DOM element of the step
//   - step: The step object configured for the step
//   - options.config: The current configuration options
//   - options.state: The current state of the driver
//   - options.driver: Current driver object
//   - options.index: Index of the currently active step
onNextClick?: (
  element?: Element,
  step: DriveStep,
  options: { config: Config; state: State; driver: Driver; index: number
) => void;
```

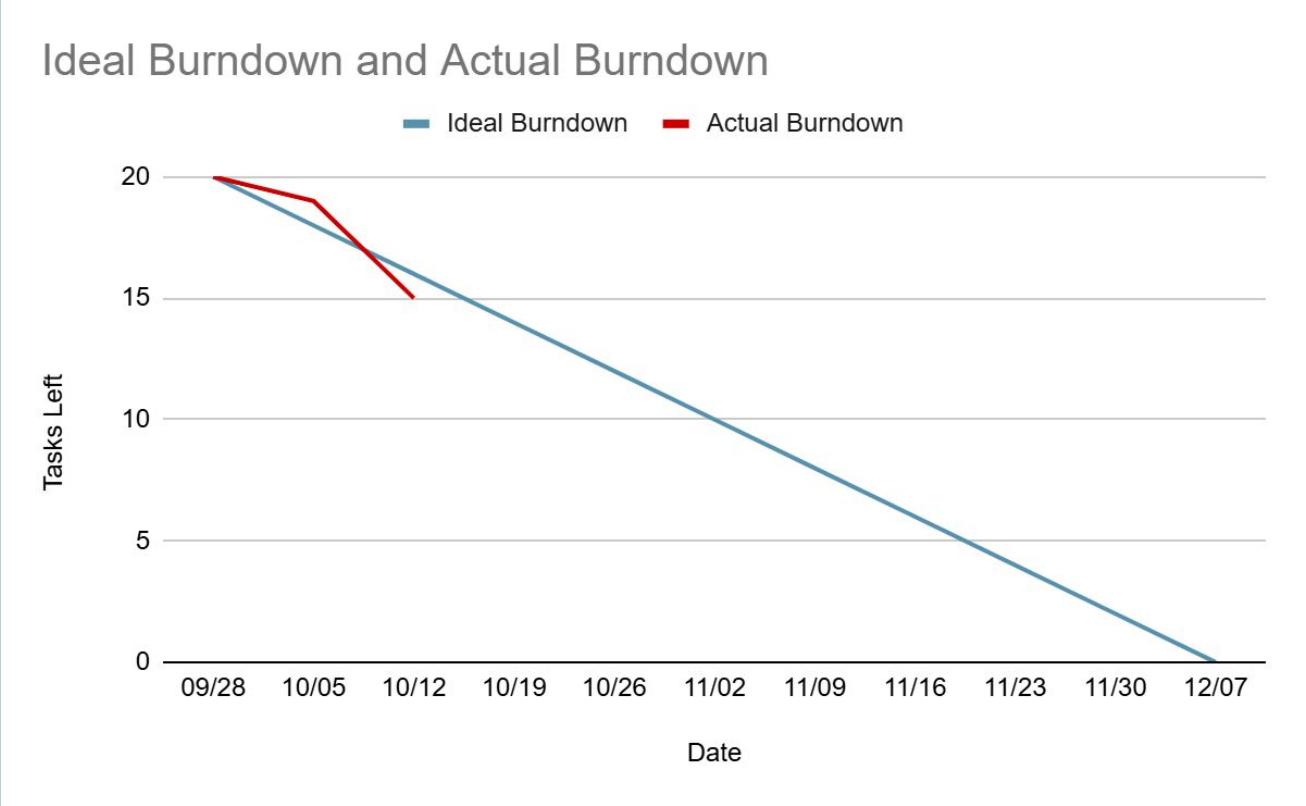
Incomplete

Continued Work

- **Written Documentation of Features**
  - More time working on features and building then documenting changes and implementation.

- **Website Interaction With Firebase**
  - Further Investigation on how to store tours and their elements with Firebase

Burndown Chart



Project Demo

Sprint #3 Projected Progress

Adjusted Goals:

Week 1:

- User action detection functional
- Completion conditions flags
- Firebase Integration

Week 2:

- Complete Reusable Dynamic tour system
- Finish Complex Additional tours

Individual Goals:

Wyatt:

-Full User Input and Object detection in Tours

-Iterate/Review on File Organization and Merges

-Further Improve UI

Sam:

- Think of ideas on how to “automate” tour creation
- Create more (complex) tours

Ben:

- Save a “completed” flag to Firebase when a tour finishes
- Confirm with sponsor if Firebase can be used this way

Jason:

- Implement logical checks in the tour system to advance automatically
- Test for any bugs

